Which results in this:

```
$ rspec
.

Finished in 0.00392 seconds (files took 5.99 seconds to load)
1 example, 0 failures
```

And the test passes! You're done! Ship it!

Okay, you're not exactly done. You made the test pass, which actually only gets you two-thirds of the way through the TDD cycle. You've done the "red" (failing test) step and the "green" (passing test) step and now you're at the refactoring step. However, you've written almost no code, so you can safely say there are no refactorings indicated at this point.

I suspect that if you're inclined to be skeptical of test-driven development, I haven't convinced you yet. I've gone on for a few pages and you've only written one line of code, and that line of code clearly isn't even final. I reiterate that in practice this doesn't take much time. If I wasn't stopping to discuss each step this would take only a couple of minutes, and some of that time—like setting up the Project class—would need to be spent anyway.

In fact, you haven't exactly done nothing—you've defined and documented a subtle part of how your Project class behaves, and you'll find out immediately if the class ever breaks that behavior. As I've said, though, documentation and regression are only part of what makes test-driven development powerful. You need to get to the design part. And for that, you need to write more tests.

The Second Test

One nice feature of test-driven development is that making one test pass often points the way to the next test. The goal of the next test cycle is to write a test that fails given the current code. At this point the code says that done? is always true, so you should create a case where done? is false:

basics/02/spec/models/project_spec.rb

```
it "knows that a project with an incomplete task is not done" do
  project = Project.new
  task = Task.new
  project.tasks << task
  expect(project.done?).to be_falsy
end
```

This test is similar to the first one, but now you have a second class, Task, and a related attribute of the Project class, tasks. This time you're assuming that a new task is undone, and therefore a project with an undone task is